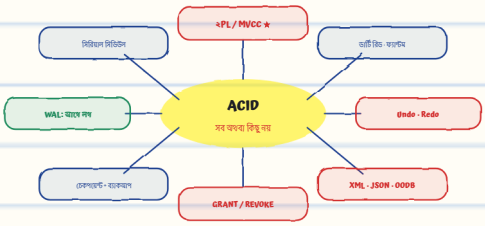
ট্রানজেকশন, কনকারেন্সি, রিকভারি ও নিরাপত্তা

ACID · ২PL · MVCC · আইসোলেশন লেভেল · WAL · GRANT · XML/JSON · OODB/ORDB

এগারো পাতা: ট্রানজেকশন ও তার চারটি বৈশিষ্ট্য, কনকারেন্সির সৃষ্টি সমস্যা, সেগুলো থামানোর চারটি কৌশল, চারটি আইসোলেশন লেভেল, লগ দিয়ে ক্র্যাশের পর ডেটাবেস কীভাবে গড়ে ওঠে, কে কী দেখতে পারে — এবং এরপরের আধা-কাঠামোবদ্ধ ও অবজেক্ট জগৎ।



↑ পুরো টপিক এক পাতায়: লাল = বেশি আসে, সবুজ = রিকভারির ইঞ্জিন

➜ সিলেবাসের যে অংশ

সিলেবাস এই লেসনের প্রতিটি অংশ নাম করে: "*Transaction processing; DBMS concurrency control; database recovery; database security and access control; XML, XPath and XQuery; object-oriented and object-relational databases.*" **প্রতিটি অংশই এখানে পূর্ণ করা হয়েছে।** আগের লেসনগুলোতে সম্পর্ক বানানো, পড়ার SQL, রিডান্ড্যান্সি সরানো ও ডিস্কে জায়গা দেওয়া হয়েছিল; এই লেসনে সেই সবকিছুকে একসঙ্গে অনেক ব্যবহারকারী, একটি ব্যর্থতা ও একজন অনুমানকারী র মুখোমুখি হতে হবে — দশজন শিক্ষার্থী একসঙ্গে লগইন করলে, একজন মাঝপথে ক্র্যাশ করলে আর একজন অন্যের বেতন দেখতে চাইলে, ডেটাবেস কি তবুও ঠিক থাকবে?

হাই-ইল্ড তালিকা

① ACID = অ্যাটমিসিটি, কনসিসটেন্সি, আইসোলেশন, ডিওরেবিলিটি
② সব অথবা কিছু নয়: COMMIT বা ROLLBACK
③ Active → পার্শিয়ালি কমিটেড → কমিটেড
④ ব্যর্থতার পরে Failed → Aborted
⑤ সিরিয়ালাইজেবল = যেন একটির পর আরেকটি
⑥ ডার্টি · লস্ট আপডেট · নন-রিপিটেবল · ফ্যান্টম
⑦ S+S যায় · S+X না · X+X কখনোই না
⑧ ২PL: আগে লক নাও, পরে ছাড়ো — ডেডলক তবুও হয়
⑨ RU < RC < RR < সিরিয়াল = নিরাপদ, ধীর
⑩ WAL নিয়ম: আগে লগ রেকর্ড, তারপর ডেটা পেজ
⑪ আনকমিটেডের UNDO, কমিটেডের REDO
⑫ চেকপয়েন্ট + ফুল / ইনক্রিমেন্টাল ব্যাকআপ

সংকেত হলুদ = মূল শব্দ · লাল বৃত্ত = অবশ্যই মুখস্থ · **বেগুনি** = মনে রাখার কৌশল · **লাল** = পরীক্ষার ফাঁদ · **সবুজ** = সমাধান-কৃত উদাহরণ

➜ সূচিপত্র

✓ ট্রানজেকশন ও ACID বৈশিষ্ট্য ... পৃ. ২
✓ সিডিউল ও কনকারেন্সি সমস্যা ... পৃ. ৩
✓ লক: S/X, ২PL ও ডেডলক ... পৃ. ৪
✓ টাইমস্ট্যাম্প, অপটিমিস্টিক, MVCC ... পৃ. ৫
✓ আইসোলেশন লেভেল, মানদণ্ড বনাম বাস্তব ... পৃ. ৬
✓ রিকভারি: লগ, WAL, চেকপয়েন্ট ... পৃ. ৭
✓ অ্যাক্সেস কন্ট্রোল ও নিরাপত্তা ... পৃ. ৮
✓ XML, JSON, OODB/ORDB ... পৃ. ৯
✓ রাতের পরীক্ষার পাতা ... পৃ. ১০
✓ অনুশীলনী MCQ + লিখিত ... পৃ. ১১

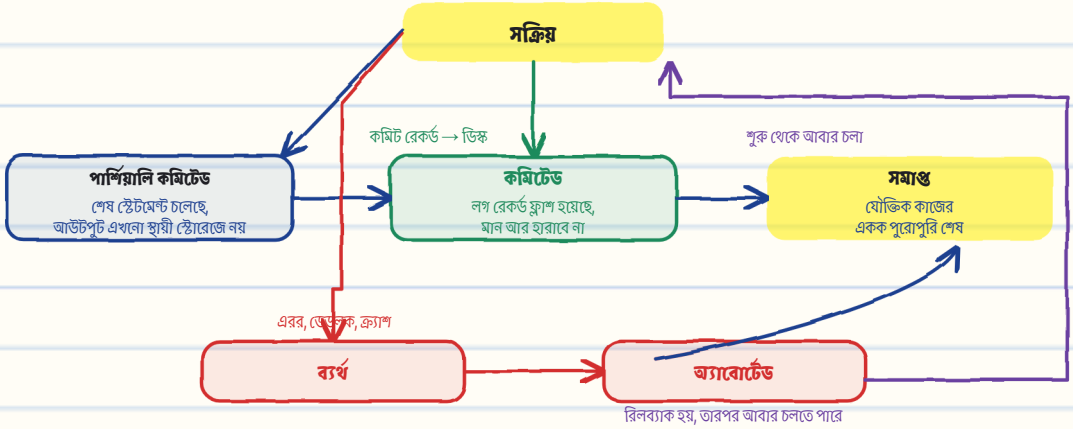
১ ট্রানজেকশন: একটি যৌক্তিক কাজের একক, ও তার চারটি বৈশিষ্ট্য

সব অথবা কিছু নয় ★

সংজ্ঞা **ট্রানজেকশন (Transaction)** হলো একটি একক যৌক্তিক কাজ হিসেবে সম্পাদিত অপারেশনের ক্রম। এক পাশে BEGIN / COMMIT, অন্য পাশে COMMIT / ROLLBACK — অর্থাৎ **সব** অপারেশন কার্যকর হবে, নয়তো **একটিও** নয়। *Tk 1000 পাঠানো মানে দুটি লেখা — অ্যাকাউন্ট A থেকে কাটা, অ্যাকাউন্ট B-তে যোগ করা — আর প্রথমটি যদি একমাত্র ডিস্কে পৌঁছায় তবে টাকা ধ্বংস হয়ে যায়; তাই দুটি লেখা একটিই ট্রানজেকশন হতে হবে।*

```
BEGIN TRANSACTION;
  UPDATE account SET balance = balance - 1000 WHERE accno = 'A-101';   -- 1
UPDATE account SET balance = balance + 1000 WHERE accno = 'B-207';   -- 2
COMMIT;   -- দুটোই, নয়তো একটিও নয়
```

বৈশিষ্ট্য	কী প্রতিশ্রুতি দেয়	কে রক্ষা করে
অ্যাটমিসিটি (Atomicity)	সব অথবা কিছু নয়। মাঝপথে ব্যর্থতা হলে পুরো একক পিছনে ফিরে যায়; অর্ধেক প্রয়োগ হওয়া ট্রান্সফার কখনো দৃশ্যমান হয় না।	রিকভারি ম্যানেজার (UNDO)
কনসিসটেন্সি (Consistency)	ঘোষিত ইন্টেগ্রিটি কনস্ট্রেন্ট ট্রানজেকশনের আগে ও পরে মান্য। **কনস্ট্রেন্টগুলো ব্যবহারকারী লেখেন** — ডিবিএমএস নিজে বানায় না।	কনস্ট্রেন্ট, ব্যবহারকারীর SQL
আইসোলেশন (Isolation)	প্রতিটি ট্রানজেকশন একা একা চলছে বলে মনে হয়। কনকারেন্সি কন্ট্রোল ঠিক এই বৈশিষ্ট্যই দেয়।	কনকারেন্সি কন্ট্রোল
ডিওরেবিলিটি (Durability)	কমিট করার পর পরিবর্তন যেকোনো ক্র্যাশেও টিকে থাকে — বিদ্যুৎ চলে গেলেও, মেশিন রিবুট হলেও।	রিকভারি ম্যানেজার (REDO, WAL)



ট্রানজেকশন-স্টেট ডায়াগ্রামের পূর্ণ চেহারা: পাঁচটি অবস্থা, আর অ্যাবোর্টেড থেকে ফেরার লুপ

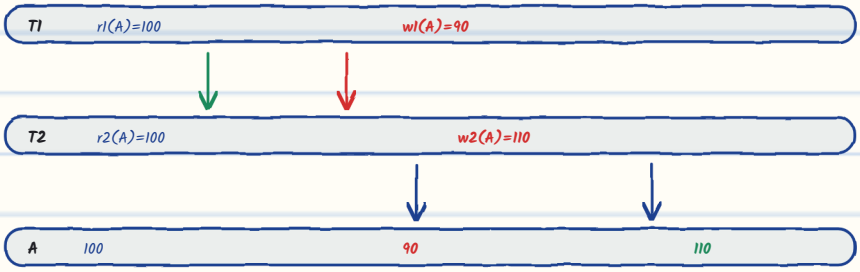
পরীক্ষার ফাঁদ ✍ পাঁচটি ভুল আঁকা ১. **ব্যর্থ (Failed)** অবস্থায় আসা হয় **সক্রিয় (Active)** থেকে — *পার্শিয়ালি কমিটেড* থেকে নয়; আংশিক অবস্থা রাখা হয়েছে যাতে আউটপুটের সময় ব্যর্থতা ধরা পড়লেও ইউন্ডো করা যায়। ২. **অ্যাবোর্টেড আসে ব্যর্থের পরে**, উল্টো নয়। ৩. **সমাপ্ত (Terminated)** হলো আসল শেষ অবস্থা; এটি আসে **কমিটেড** *এবং* **অ্যাবোর্টেড** — থেকেও — "কমিটেডই শেষ অবস্থা" ভুল। ৪. অ্যাবোর্টেড ট্রানজেকশন শুরু থেকে আবার চালানো যেতে পারে। ৫. **কমিট** মানে *লগ রেকর্ড স্থায়ী স্টোরেজে* — এর মানে নয় যে সব নোংরা পেজ ফ্লাশ হয়ে গেছে।

মনে রাখার কৌশল **"কমিটির মতো ভাবো: অ্যাটমিক কখনো ভাঙে না, কনসিসটেন্ট নিয়ম মানে, আইসোলেটেড নিজের পরামর্শে চলে, ডিওরেবল বেরিয়ে গেলেও টিকে থাকে।"** আর ডায়াগ্রামের জন্য: **"সক্রিয় দিয়ে শুরু, পার্শিয়ালি দরজা, কমিটেড প্রতিশ্রুতি, সমাপ্ত দরজা বন্ধ; কেবল ব্যর্থতাই পাশের দরজা খুলে অ্যাবোর্টেডে নিয়ে যায়।"**

২ সিডিউল, সিরিয়ালাইজেবিলিটি ও পাঁচটি অসঙ্গতি

যেন একটির পর আরেকটি ★

সংজ্ঞা **সিডিউল (Schedule)** হলো কয়েকটি ট্রানজেকশনের অপারেশন কী ক্রমে আপনাহিত হয়েছে। **সিরিয়াল সিডিউল**-এ একটি ট্রানজেকশন পুরো শেষ হয়, তারপর পরেরটি শুরু — সবসময় সঠিক, বাস্তবে অকার্যকর। **কনকারেন্ট সিডিউল** থ্রুপুটের জন্য অপারেশন মেশায়, আর তা **সিরিয়ালাইজেবল** যদি কোনো একটি সিরিয়াল সিডিউলের সমতুল্য হয় — পুরো কনকারেন্সি কন্ট্রোলের লক্ষ্য এই একটি শব্দ।



এক সিডিউল, দুটি ব্রুটি: r2(A) পড়ে নেয় এমন মান যা T1 কখনো কমিট করেনি

অসঙ্গতি	ঠিক কী ভুল হয়	কী সারায়
ডার্টি রিড (Dirty Read)	T2 এমন আইটেম পড়ে যা T1 লিখেছে কিন্তু কমিট করেনি; এরপর T1 রোলব্যাক করলে T2-এর উত্তর দাঁড়ায় এমন মানের উপর, যা কখনো ছিল না।	S→X সংঘর্ষ · RR+
লস্ট আপডেট (Lost Update)	দুজনেই A=100 পড়ে, তারপর দুজনেই লেখে; দ্বিতীয় লেখা প্রথমটিকে দেখেই উপরে লেখে দেয়। দুজন কমিট করলেও ভুল মান টেকে থাকে।	কমিট পর্যন্ত X লক · ডিলেড লেখা
নন-রিপিটেবল রিড	একই রিড কোয়েরি একটি ট্রানজেকশনের ভেতরে দুবার ভিন্ন মান দেয়, কারণ মাঝে অন্য ট্রানজেকশন সারি আপডেট করেছে।	কমিট পর্যন্ত S লক · RR+
ফ্যান্টম রিড	পুনরাবৃত্ত রেঞ্জ কোয়েরিতে নতুন বা মুছে যাওয়া সারি দেখা যায়। কোনো মান বদলায়নি, সারির অস্তিত্ব বদলেছে।	রেঞ্জ লক · সিরিয়ালাইজেবল
অসামঞ্জস্যপূর্ণ বিশ্লেষণ (Incorrect Summary)	একটি অ্যাগ্রিগেট আপডেটের আগে টেবিলের অংশ, পরে বাকিটা পড়ে, ফলে মোটটা কোনো মুহূর্তেই সত্যি ছিল না।	পঠিত প্রতিটি সারিতে S লক

পরীক্ষার ফাঁদ ✍ চারটি **১. লস্ট আপডেট সবচেয়ে বিপজ্জনক, কারণ দুটো ট্রানজেকশনই কমিট করে** — কোনো রোলব্যাকে তা ফেরানো যায় না। **২. আইসোলেশন লেভেলের মানদণ্ড-সারণায় লস্ট আপডেট নেই**, ওখানে শুধু রিড-সংক্রান্ত অসঙ্গতি আছে। **৩. ফ্যান্টম মানে সারির আগমন, মান বদলানো নয়** — মান বদলালে সেটা নন-রিপিটেবল রিড। **৪.** "কনকারেন্সি কন্ট্রোল কেবল ডার্টি রিড নিয়ে" বাক্যাংশ অতিরঞ্জিত; সব চারটির সঙ্গে অসামঞ্জস্যপূর্ণ বিশ্লেষণ মিলিয়ে মানে প্রশ্নটি মানক পাঁচটির সেট চাইছে।

মনে রাখার কৌশল "**ডার্টি = অতি তাড়াতাড়ি, লস্ট = অতি দেরিতে, নন-রিপিটেবল = মান বদলাল, ফ্যান্টম = সারি এল।**" দুটি পড়া নিয়ে, দুটি লেখা নিয়ে; "ফ্যান্টম ঘটে মান বদলানোর সময়" — এই বাক্যাংশ কিছুই বর্ণনা করে না।

উত্তরের আরেক অর্ধেক **সিরিয়ালাইজেবিলিটি প্রোগ্রামের নয়, সিডিউলের গুণ** — একই ট্রানজেকশনের দুটি সিডিউল সঠিকতায় ভিন্ন হতে পারে। তাই ডিবিএমএস আপনাহিত বন্ধ করে না; সে এমন যেকোনো আপনাহিত গ্রহণ করে যার পরিণতি সিরিয়াল সিডিউল থেকে আলাদা নয়।

৩ লক-ভিত্তিক কন্ট্রোল: সামঞ্জস্য ম্যাট্রিক্স, 2PL ও ডেডলক

সবচেয়ে বেশি পরীক্ষিত সারণি ★

দুটি লক আইটেম পড়তে **শেয়ার্ড লক** *S* নেওয়া হয় — অনেক ট্রানজেকশন একসঙ্গে ধরে রাখতে পারে। লিখতে **এক্সক্লুসিভ লক** *X* নেওয়া হয় — একটির বেশি পারে না। আইটেমের নাম *X* রাখা হয় কারণ লক **বদলে নয়, যোগ করে** উন্নীত হয়।

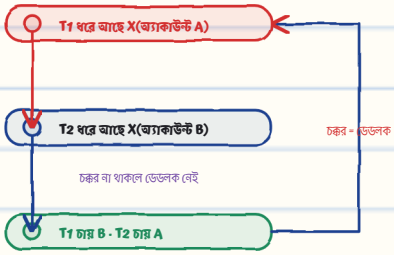
ধারিত \ চাওয়া	*S*(A)	*X*(A)
S(A) ধারিত	সামঞ্জস্যপূর্ণ ✓	সংঘর্ষ ✗
X(A) ধারিত	সংঘর্ষ ✗	সংঘর্ষ ✗

পরীক্ষার ফাঁদ ✍ তিনটি সিদ্ধান্ত সবাই ভুল করেন ১. চার ঘরের মধ্যে **একটিই** সামঞ্জস্যপূর্ণ — ***S*-এর সঙ্গে *S***। "পড়ার জন্য *S*-এর সঙ্গে *X* সামঞ্জস্যপূর্ণ" মিথ্যা, কারণ *X* ধারী তখন মান বদলাতে পারে। ২. *X* চাওয়া দুটি ট্রানজেকশন কখনো মিলিত হয় না, তাই *2PL*-এ রিড-রাইট অসঙ্গতি টেকে না; বিনিময়ে দিতে হয় ব্লকিং। ৩. *S* ধরে *X* চাওয়া ট্রানজেকশন **সরাসরি *X* নিতে পারে না** — আগে *S* ছেড়তে হয়, আর *2PL*-এ তখন সংক্ষেপণ পর্বই শেষ হয়ে গেছে হতে পারে।

টু-ফেজ লকিং (2PL) ✍ সংজ্ঞা প্রতিটি ট্রানজেকশন **দুটি পর্বে** চলে: গ্রোয়িং পর্ব, যেখানে লক **অর্জন** করা যায় কিন্তু ছাড়া যায় না; আর শ্রিংকিং পর্ব, যেখানে লক **ছাড়া** যায় কিন্তু নতুন করা যায় না। সংক্ষেপণ পর্বের শেষে এটি কমিট করে।

পরীক্ষার ফাঁদ ✍ 2PL কী দেয়, কী দেয় না **দেয়:** কনফ্লিক্ট-সিরিয়ালাইজেবল সিডিউল (প্রিসিডেন্স গ্রাফে চক্কর হয় না); কিন্তু নিজে থেকে **স্ট্রিক্ট** বা **রিকভারেবল** সিডিউলের নিশ্চয়তা **দেয় না** — কমিটের আগেই লক ছাড়া যায়, ফলে ক্যাসকেডিং রোলব্যাক হতে পারে; তা আটকায় **স্ট্রিক্ট *2PL***। **দেয় না:** ডেডলক থেকে মুক্তি বা স্টারভেশন থেকে মুক্তি — বিপরীত ক্রমে লক নেওয়া দুটি ট্রানজেকশন *2PL*-এ ডেডলক করে, আর এটিই এই টপিকের সবচেয়ে বেশি ভুল দাবি।

মনে রাখার কৌশল "**আগে শুধু দরজা খুলবে, পরে শুধু দরজা বন্ধ করবে।**" **স্ট্রিক্ট *2PL*** প্রতিটি *X* লক কমিট পর্যন্ত ধরে রাখে — স্ট্রিক্ট সিডিউল, তাই **ক্যাসকেডিং রোলব্যাক নেই**; **রিগরাস *2PL* সব** লক (*S* ও *X*) কমিট পর্যন্ত ধরে রাখে। **কনজারভেটিভ (স্ট্যাটিক) *2PL*** প্রয়োজনীয় প্রতিটি লক **শুরুর আগেই** ঘোষণ করে, তাই কোনো লক না পেলে ট্রানজেকশনই শুরু হয় না: **ডেডলক-মুক্ত**, তবে কনকারেন্সির খরচ অনেক।



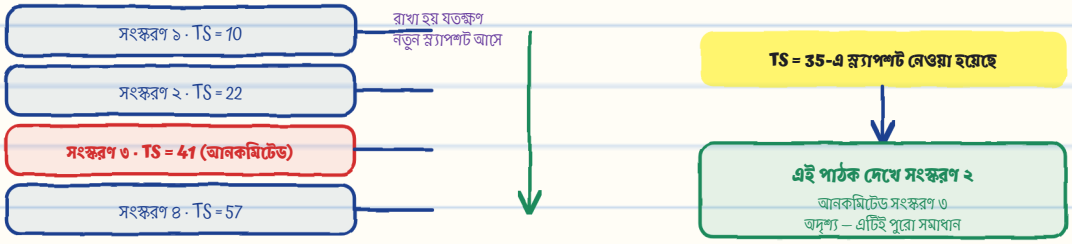
ওয়েট-ফর গ্রাফ: চক্কর মানেই ডেডলক, আর *2PL* তা বাদ দেয় না

ডেডলক সামলানোর উপায়	পদ্ধতি
প্রতিরোধ (*Prevention*) — ঘটার আগেই	কোন আইটেমে প্রথম লক নেওয়া যাবে তার একটি **নির্দিষ্ট বৈশ্বিক ক্রম** ঠিক করে দাও — তাহলে চক্কর অসম্ভব। ডেটা-ইঞ্জিনিয়ারিংয়ে বিকল্প হলো ***wait-die*** ও ***wound-wait***, যেগুলো টাইমস্ট্যাম্প ব্যবহার করে।
শনাক্তকরণ (*Detection*) — ঘটার পরে	একটি **ওয়েট-ফর গ্রাফ** রাখো: $T_i \to T_j$ মানে T_i T_j-এর জন্য অপেক্ষা করছে। নিয়মিত খুঁজে দেখো **চক্কর** আছে কি না; পেলে চক্করের **একটি ট্রানজেকশন অ্যাবোর্ট** করো — সাধারণত সবচেয়ে কম কাজ করা ট্রানজেকশনটি — তার লক ছাড়ো এবং আবার চালানোর সুযোগ দাও।

৪ ২PL-এর পর: টাইমস্ট্যাম্প অর্ডারিং, অপটিমিস্টিক কন্ট্রোল ও MVCC

লক নেই, তবু সিরিয়ালাইজেবল ★

কৌশল	কীভাবে কাজ করে	আটকাটা
টাইমস্ট্যাম্প অর্ডারিং (TO)	প্রতিটি ট্রানজেকশন পায় TS(T) = (সময়, গণনা), যাতে **TS অদ্বিতীয় ও তুলনীয়** হয়। কোনো পড়া বা লেখা তখনই বৈধ যখন সেটি আইটেমের শেষ লেখকের সময় মেনে চলে; নয়তে ট্রানজেকশনটি অ্যাবোর্ট ও পুনরায় চালাতে হয়।	ডেডলক-মুক্ত, তবে **কাজ নষ্ট হতে পারে** এবং স্টারভেশন হতে পারে।
অপটিমিস্টিক (ভ্যালিডেশন)	ধরে নাও দ্বন্দ্ব **বিরল**: একেবারে লক ছাড়া চালাও, পঠিত ও লেখা আইটেম মনে রাখো, আর কমিটের সময় **যাচাই (validate)** করো যে পঠিত কিছু বদলায়নি।	**লেখা-নির্ভর** কাজের জন্য অনুপযুক্ত — ভ্যালিডেশনের উত্তর প্রায়ই "অ্যাবোর্ট"।
MVCC (মাল্টিভার্সন)	প্রতিটি সারির কয়েকটি সংস্করণ রাখো, যেগুলোতে লেখার ট্রানজেকশনের ছাপ আছে। পাঠক একটি **স্ন্যাপশট** নিয়ে যে সংস্করণ দেখা উচিত তাই পড়ে, তাই **পাঠক লেখককে আটকায় না**; শুধু একই সারিতে দুই লেখকই সংঘর্ষ করে।	পুরোনো সংস্করণ সরাতে হয়, আর **রাইট-স্কেউ** কমিটের সময়ই ঠেকাতে হয়।



এক সারি, চারটি জীবন্ত সংস্করণ — পাঠকের স্ন্যাপশট টাইমস্ট্যাম্পই বেছে নেয়

MVCC: সারির কয়েকটি সংস্করণ থাকে; পাঠকের স্ন্যাপশট টাইমস্ট্যাম্প ঠিক করে কোনটি দেখবে

পরীক্ষার ফাঁদ ✍ MVCC মানে "কোনো লক নেই" নয় ✍ তিনটি

১. "*MVCC* একেবারে লক ব্যবহার করে না" মিথ্যা — পাঠক কোনো লক নেয় না, তবে একই সারিতে **দুজন লিখলে** সংঘর্ষ হয়। ২. এর বিক্রয়-বিন্দু প্রায়ই ভুল বলা হয়: এটি **পাঠক-লেখক** ব্লকিং দূর করে, রাইট-রাইট সংঘর্ষ দূর করে না। ৩. "*MVCC* সব অসঙ্গতি ঠেকায়" মিথ্যা — স্ন্যাপশট আইসোলেশনেও **রাইট-স্কেউ** ঢোকে, তাই আসল ইঞ্জিন তা ধরে সিরিয়ালাইজেশন ব্যর্থতা দিয়ে অ্যাবোর্ট করে, যা ক্লায়েন্ট আবার চালায়।

কোথায় কোনটি ব্যবহৃত হয় ***PostgreSQL*** পুরোনো টিউপল রাখে এবং VACUUM দিয়ে ফেরায়; ***Oracle*** রোলব্যাক সেগমেন্টে *undo* রাখে এবং তা দিয়ে রিড-কনসিসটেন্ট স্ন্যাপশট গড়ে; ***MySQL InnoDB*** *undo* লগ রাখে ও আনকমিটেড সংস্করণ অন্য সেশনকে দেখায় না; ***SQL Server*** *tempdb*-তে সারি-সংস্করণ রাখে। লক ও সংস্করণ প্রতিদ্বন্দ্বী নয় — এই প্রতিটি সিস্টেম দুটোই ব্যবহার করে।

মনে রাখার কৌশল **"লক সার পড়ে, টাইমস্ট্যাম্প বিচার করে, অপটিমিস্ট আশা করে, সংস্করণ চালচা করে।"** — *২PL* ট্রানজেকশনকে পরস্পরের *অপেক্ষা* করায়, টাইমস্ট্যাম্প অর্ডারিং দেরি করা ট্রানজেকশনকে *আবার চালায়*, অপটিমিস্ট কন্ট্রোল তাকে *আশা* করতে বলে যে কিছুই বদলায়নি, আর *MVCC* সবাইকে পুরোনো কপি দেখায়, ফলে কেউ অপেক্ষাই করে না।

৫ আইসোলেশন লেভেল: SQL মানদণ্ড, এবং আসল ইঞ্জিন যা করে

একটি ডায়াল, চার অবস্থান ★

সংজ্ঞা **আইসোলেশন লেভেল (Isolation Level)** হলো একটি ট্রানজেকশনের কাজ অন্যটির চোখে কতটা লুকানো থাকে — অর্থাৎ কনসিসটেন্সি বনাম থ্রুপুট-র ডায়াল। সবচেয়ে শক্তিশালী নিশ্চয়তার জন্য ডানে ঘোরালে গতি সবচেয়ে কম, বামে ঘোরালে গতি বেশি কিন্তু অসঙ্গতি মেনে নিতে হয়। সর্বোচ্চ নিচের প্রতিটি লেভেল **কোন কোন অসঙ্গতি অনুমতি দেয়** তা দিয়েই সংজ্ঞায়িত।

আইসোলেশন লেভেল	ডার্টি রিড	নন-রিপিটেবল	ফ্যান্টম	গতি
রিড আনকমিটেড	অনুমোদিত	অনুমোদিত	অনুমোদিত	সবচেয়ে দ্রুত
রিড কমিটেড	না	অনুমোদিত	অনুমোদিত	দ্রুত
রিপিটেবল রিড	না	না	অনুমোদিত	ধীর
সিরিয়ালাইজেবল	না	না	না	সবচেয়ে ধীর

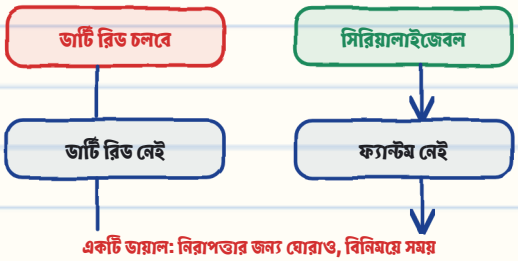
```
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
BEGIN TRANSACTION;
  SELECT balance FROM account WHERE accno = 'A-101';
  -- একই SELECT আরেকবার এখানে একই মান দিতে হবে
COMMIT;
```

পরীক্ষার ফাঁদ ✍ মানদণ্ডের সারণি প্রোডাক্ট যা করে তা নয় ✍ চারটি

১. উপরের সারণি **SQL মানদণ্ডের** সংজ্ঞা, যেখানে রিপিটেবল রিডেও ফ্যান্টম অনুমোদিত — এটি সারির *মান* বদলানো আটকায়, সারির *আগমন* নয়। ২. **MySQL InnoDB** আরও এগোয়: next-key (গ্যাপ + রেকর্ড) লক ব্যবহার করে রিপিটেবল রিডেও ফ্যান্টম আটকায়। ৩. **PostgreSQL** রিপিটেবল রিডকে স্ন্যাপশট আইসোলেশন হিসেবে বাস্তবায়ন করে, যেটিও ফ্যান্টম আটকায়, আর সত্যিকারের সিরিয়ালাইজেবল লেভেলে রাইট-স্কেউ-ও যাচাই হয়। ৪. **SQL Server** মানদণ্ডের কাছাকাছি থাকে, তাই রিড রিপিটেবলেও ফ্যান্টম সম্ভব। তাই "রিপিটেবল রিড ফ্যান্টম আটকায়" কথাটি **ইঞ্জিন-নির্ভর** — দ্ব্যর্থহীন MCQ-র প্রিয় ফাঁদ।

লস্ট আপডেট: যা সারণি দেখাতে পারে না তিনটি কলামই *পড়া* নিয়ে। **লস্ট আপডেট** একটি *লেখার* সংঘর্ষ, আর মানদণ্ড লেভেল বাড়িয়ে তা দূর করে না — অনেক ইঞ্জিনে একই লেভেলেও সারি লক না করলে আপডেট হারায়, তবে অ্যাপ্লিকেশন ডিলেড (অপটিমিস্টিক) লেখা করতে পারে: মান পড়ে নাও, তারপর `WHERE balance = <যে মান পড়েছিলে>` দিয়ে লেখো; কিছু না মিললে আবার চেষ্টা করো।

মনে রাখার কৌশল **"রিড আনকমিটেড গুজব, রিড কমিটেড সংবাদ, রিপিটেবল রিড সেদিনের দলিল, সিরিয়ালাইজেবল আদালতের লিপিবদ্ধ কার্যবিবরণী।"**
সিঁড়িটি এক বাক্যে:
~~বেশি আইসোলেশন = কম অসঙ্গতি = বেশি অপেক্ষা~~।
লিখিত উত্তরে সবচেয়ে শক্তিশালী বাক্য: "লেভেল বাড়ালে যৌক্তিক ত্রুটির বদলে ভৌত অপেক্ষা আসে, তাই নির্বাচনটি সঠিকতা ও থ্রুপুটের মধ্যে।"



সিঁড়িটি ডার্টি রিড থেকে সিরিয়াল সিডিউল পর্যন্ত

৬ রিকভারি: লগ, WAL নিয়ম, UNDO/REDO, চেকপয়েন্ট ও ব্যাকআপ

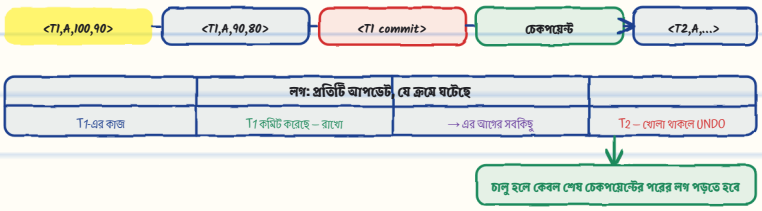
ক্র্যাশের পাতা ★

ব্যর্থতা	যা ঘটেছে	যা সামলায়
ট্রানজেকশন ব্যর্থতা	লজিক্যাল এরর, কনস্ট্রেন্ট লঙ্ঘন, ডেডলকের শিকার, বা ব্যবহারকারী নিজে *abort*।	সরাসরি **UNDO** — ডিবিএমএস আবার চালু হয় না
সিস্টেম ক্র্যাশ	বিদ্যুৎ চলে যাওয়া, OS বন্ধ, হার্ডওয়্যার বিভ্রাট। র‍্যাম গেছে; **যা ডিস্কে ছিল তা বেঁচে গেছে**।	লগ থেকে **REDO**
মিডিয়া / ডিস্ক ব্যর্থতা	খারাপ সেক্টর, হেড ক্র্যাশ, ভলিউম হারানো। ডেটাবেস নিজেই ক্ষতিগ্রস্ত বা নেই।	**ব্যাকআপ** থেকে *restore* + লগের **REDO**

লগ রেকর্ডে থাকে কী প্রতিটি রেকর্ডে থাকে **ট্রানজেকশন আইডি**, **আগের লগ-রেকর্ডের ঠিকানা** (তাই লগ দুই দিকেই সাধানিত তালিকা — রোলব্যাকের জন্য পিছনে, রিডোর জন্য সামনে), **ডেটা আইটেম**, তার **পুরোনো মান** (*before-image*) ও **নতুন মান** (*after-image*), এবং `<T_i, X, V_old, V_new>` কমিট/অ্যাবোর্ট চিহ্ন। একটি **চেকপয়েন্ট** নিজেও একটি লগ রেকর্ড — ডেটাবেসের সম্পূর্ণ কপি নয়।

পরীক্ষার ফাঁদ ✍ WAL নিয়ম ✍ তিনটি **রাইট-অ্যাহেড লগিং নিয়ম:** **আপডেটের লগ রেকর্ড স্থায়ী স্টোরেজে পোছানোর আগে সংশ্লিষ্ট ডেটা পেজ স্থায়ী স্টোরেজে পৌঁছাতে পারে না।** ১. "ডেটাবেসে লেখার পরে লগ" নিয়মটি উল্টে দেয়, তাই যেকোনো *MCQ*-তে ভুল অপশন। ২. *WAL*-ই **ডিওরেবিলিটি ও অ্যাটমিসিটি** সস্তা করে; এটি **ব্যাকআপ নয়** এবং হারানো ডিস্ক রক্ষা করে না। ৩. "চেকপয়েন্ট পুরো ডেটাবেস কপি করে" এটি ভুল ধারণা — একটি **ফাজি চেকপয়েন্ট** কেবল নোংরা পেজগুলোর নাম লিখে রাখে, তাই কোনো কাজ আটকে না।

আপডেট পলিসি	ডেটা পেজ কখন লেখা হয়	ক্র্যাশের পরে আমাদের...
ডিফার্ড আপডেট	শুধু **কমিটের পরে** — আগে কোনো পেজ স্পর্শ হয় না।	শুধু **REDO** — ডিস্কে আনকমিটেড কিছু নেই।
ইমিডিয়েট আপডেট	বাফার ফ্লাশ হওয়ামাত্র, **কমিটের আগেই**।	আনকমিটেডের **UNDO**, কমিটেডের **REDO**।



একটি লগ, একটি চেকপয়েন্ট: রিকভারি কেবল শেষ চেকপয়েন্টের পরের অংশই চালায়

ব্যাকআপ ও বাকি কৌশল ✍ **ফুল** — সব কপি; ধীর, কিন্তু *restore* দ্রুত। **ডিফারেনশিয়াল** — **শেষ ফুল ব্যাকআপের পর** যা বদলেছে; সময়ের সঙ্গে বাড়ে। **ইনক্রিমেন্টাল** — **যেকোনো শেষ ব্যাকআপের পর** যা বদলেছে; সবচেয়ে ছোট, তবে *restore*-এ পুরো ধারাটি ক্রমে লাগে। **শ্যাডো পেজিং** প্রতিটি পেজের একটি ছায়া কপি রাখে এবং কমিটের সময় পরিবর্তনীয়ভাবে (*atomically*) স্থানান্তর করে — রোলব্যাকের জন্য লগ লাগে না — তবে পুরোনো পেজ-শৃঙ্খল আবর্জনা হয়ে থাকে। ***ARIES***, যা আসল সিস্টেম ব্যবহার করে, তিনটি পাস চালায়: **বিশ্লেষণ** শেষ চেকপয়েন্ট থেকে, **রিডো** (ইতিহাস পুনরাবৃত্তি), **আনডু** (ব্যর্থদের রোলব্যাক)।

মনে রাখার কৌশল **"আগে লগ, পরে ডেটা"** — এই তিন শব্দেই *WAL* নিয়ম, এবং লগ থাকার পুরো কারণ। আর **"আনকমিটেডের *UNDO*, কমিটেডের *REDO*"** — প্রতিটি ক্র্যাশের ক্ষেত্রে ধরে; ক্র্যাশের আগে যে ট্রানজেকশন কমিটও করেনি অ্যাবোর্টও করেনি তাকে *অ্যাবোর্ট হয়েছে* ধরা হয়, তাই **বিদ্যুৎ চলে যাওয়ার পরে কোনো সন্দেহ থাকে না।**

৭ অ্যাক্সেস কন্ট্রোল ও নিরাপত্তা: কে কী দেখতে পারে

DAC · MAC · RBAC ★

দুই অংশ **প্রমাণীকরণ (Authentication)** জিজ্ঞেস করে "আপনি কে?" — পাসওয়ার্ড, বায়োমেট্রিক, ওয়ান-টাইম কোড, দ্বিতীয় ফ্যাক্টর। **অনুমোদন (Authorisation)** জিজ্ঞেস করে "আপনি কী করতে পারেন?" — পড়া, ইনসার্ট, আপডেট, ডিলিট, নাকি কিছু নির্দিষ্ট কলামের কিছু সারি। ডেটাবেস **নিরাপদ** তখনই, যখন দুটোই আছে: খোলা দরজার পেছনে নিখুঁত অনুমোদন-ব্যবস্থার মূল্য নেই।

মডেল	অ্যাক্সেস কে ঠিক করে	এর বিশেষত্ব ও দুর্বলতা
DAC — ডিসক্রিশনারি অ্যাক্সেস কন্ট্রোল	প্রতিটি অবজেক্টের **মালিক**। প্রত্যেক ব্যবহারকারী নিজের অধিকার আরও এগিয়ে দিতে পারে।	নমনীয় ও সস্তা, তবে ব্যবহারকারী নিজের ধারিত অধিকারই অন্যকে দিয়ে দিতে পারে — এটিই ট্রোজান হর্সের পথ।
MAC — ম্যান্ডেটরি অ্যাক্সেস কন্ট্রোল	**সিস্টেম**, একটি সিকিউরিটি লেবেল থেকে: শ্রেণিবিভাগ × কম্পার্টমেন্ট (*Top Secret, Secret, Confidential, Unclassified*)।	কেউ শ্রেণিবিভাগ **নামাতে পারে না**, মালিকও নয় — সামরিক ও সরকারি ডেটার জন্য।
RBAC — রোল-ভিত্তিক অ্যাক্সেস কন্ট্রোল	**রোল**: অনুমতি কাজের ধরনের সঙ্গে যুক্ত, আর ব্যবহারকারীরা রোলে নিয়োগ করা হয়।	"কে এটা করতে পারে?" বিস্তার ঘটায় — চাকরি বদলালে রোল বদলাও, অনুমতির তালিকা ছুঁতে হয় না।

```
CREATE ROLE Auditor;
GRANT SELECT ON Employees TO Auditor;
GRANT UPDATE (salary) ON Employees TO Auditor;
REVOKE SELECT ON Payslips FROM intern;
```

ঝুঁকি ও তার উত্তর **স্টোরেজে** — ডিস্ক বা ব্যাকআপ চুরি: **এনক্রিপশন**। **ট্রানজিটে** — নেটওয়ার্ক ট্যাপ: **TLS**। **অসতর্ক ব্যবহারকারী**: ভিউ, রোল, লিস্ট-প্রিভিলেজ। **অসতর্ক ডেভেলপার**: *SQL* ইনজেকশন — তাই কোয়েরি **প্যারামিটারাইজড** হয়, ইনপুট জোড়া লাগিয়ে কখনো সাজানো হয় না। **ভেতরের মানুষ**: **অডিট ট্রেইল**, যাতে পরে জবাবদিহি থাকে। **অসতর্ক প্রশাসক**: *DBA* সংজ্ঞানতই সর্বোচ্চ ক্ষমতাসম্পন্ন, তাই দায়িত্ব আলাদা করা হয় — রিকভারি প্রশাসন সাধারণ *DDL* থেকে ভাগ করা থাকে।

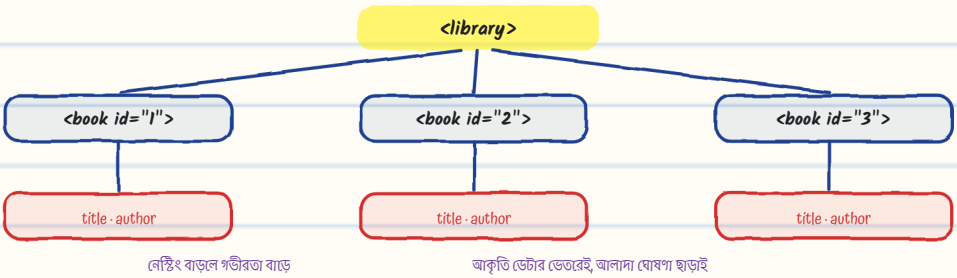
পরীক্ষার ফাঁদ ✍ পাঁচটি **১. *GRANT*** অধিকার দেয়, ***REVOKE*** কেড়ে নেয় — দুটি বিনিময়যোগ্য নয়, আর REVOKE-এ গ্রান্টিপত্রের নাম লাগে। **২. *MAC*-এ** অধিকার ঠিক করে **সিস্টেম**, তাই ব্যবহারকারী কিছুই গ্রান্ট করতে পারে না; ***DAC*-এ** অধিকার মালিকের সঙ্গে থাকে। **৩. *RBAC* রোল নিয়ে, সারি নিয়ে নয়** — সারি-ভিত্তিক অ্যাক্সেস আলাদা সুবিধা। **৪. ভিউ (*View*)** সস্তা কিন্তু শক্তিশালী দ্বিতীয় সুরক্ষা: প্রয়োজনমতো কলাম ও সারি দেখিয়ে ভিউতে *GRANT* করো, টেবিলে নয়। **৫. এনক্রিপশন** গোপনীয়তা, **অডিটিং** জবাবদিহি, **ইন্টেগ্রিটি কনস্ট্রেন্ট** সঠিকতা — তিনটি আলাদা কাজ, আর পরীক্ষা এগুলো গুলিয়ে দিতে পছন্দ করে।

লিস্ট-প্রিভিলেজ, বাস্তবে ✍ কাজ করার জন্য যেটুকু **সবচেয়ে কম** অধিকার দরকার, এর বেশি কিছুই দেবেন না: দর্শকারীর জন্য শুধু পড়ার অধিকার, অপারেটরের জন্য একটি টেবিল, পুরো টেবিল নয় GRANT UPDATE(column) যখন মাত্র একটি ফিল্ড বদলায়, আর পে-রোল টেবিলে কখনো SELECT * নয়। দেওয়া প্রতিটি অধিকার সেই মুহূর্তেই প্রতিটি ব্যাকআপ, রেপ্লিকা ও অডিট লগে চলে যায়।

মনে রাখার কৌশল "**DAC মালিক, MAC লেবেল, RBAC পদবি।**" এই ক্রমে বললে তিনটিই আটকে যায়। আর "**লিস্ট-প্রিভিলেজই পুরো নীতি**" — যে অধিকার আপনি দেননি, সেটি এমন একটি ফাটল যা ঠিক করতে হলো না।

৮ আধা-কাঠামোবদ্ধ ডেটা: XML, XPath, XQuery, JSON — ও OODB বনাম ORDB

শেষ দুটি অংশ ★



XML স্ব-বর্ণনামূলক: আকৃতি ডেটারই অংশ

দুটি কোয়েরি ভাষা — গুলিয়ে ফেলবেন না *XPath* একটি ছোট পাথ-ও-প্রেডিকেট ভাষা, একটি নথির উপর: `/library/book/title`, `/library/book[@id="2"]/author`, `//author[text()="CLRS"]`। ***XQuery*** একটি সম্পূর্ণ কোয়েরি ভাষা, *FLWOR* সহ — `for`, `where`, `order by`, `return` — এক বা একাধিক নথির উপর, এবং গণনা ও জয়েনও করতে পারে। "*XPath* দুটি নথি জয়েন করে" — এই অপশন সরাসরি ভুল।

```
for $b in /library/book
where $b/author = "CLRS"
order by $b/title
return $b/title/text()
```

রিলেশনাল, XML, JSON পাশাপাশি রিলেশনাল: *কাঠামোবদ্ধ*, আগে স্কিমা ঘোষিত, মানের সারি, জয়েন ও সেট অপারেশন কিনে পাওয়া যায়, তবে আকৃতি বদলালে স্কিমাও বদলাতে হয়। ***XML***: *আধা-কাঠামোবদ্ধ*, স্ব-বর্ণনামূলক, নেস্টিং ও বিভিন্ন ধরনের রেকর্ড, যাচাই হয় ***DTD*** বা ***XSD*** দিয়ে। ***JSON***: তাও আধা-কাঠামোবদ্ধ, *XML*-এর চেয়ে হালকা, ওয়েবে স্বাভাবিক, প্রায় সব *API*-তে ব্যবহৃত। তিনটিকেই ডিবিএমএসের উত্তর এক: নথিটি সম্পূর্ণ অবস্থায় রাখো, নাকি সম্পর্কে ভেঙে *(shred)* রাখো — "*XML* থেকে রিলেশনাল ম্যাপিং" মানে এটিই।

```
{"library":{"books":[
  {"id":1,"title":"DB Systems","author":"Navathe"},
  {"id":2,"title":"Intro to Algorithms","author":"CLRS"}]}}
$.library.books[*].title
```

পরীক্ষার ফাঁদ ✍ তিনটি ১. *XPath*-এ **অ্যাট্রিবিউট হলো টেক্সট**: প্রেডিকেট `[@id="2"]` স্ট্রিং `"2"`-এর সঙ্গে মেলে, তাই `[@id=2]` ভুল বানান। ২. `//` মানে "নিচের যেকোনো জায়গায়", "দ্বিতীয় স্তরে" নয়। ৩. *JSON*-এ কমেন্ট নেই, ডিফল্টভাবে স্কিমা নেই, আর *JSONPath* *XPath*-এর চেয়ে অনেক সরু: নেভিগেট ও ফিল্টার করতে পারে, কিন্তু *FLWOR* বা জয়েন নেই।

বিষয়	*OODB*	*ORDB*
ডেটা মডেল	**শুধু অবজেক্ট**, পরিচয় সহ	টেবিল + টাইপ
ভাষা	*OQL / ODL*	*SQL* + এক্সটেনশন
উদাহরণ	*ObjectStore, db4o, ObjectDB*	*PostgreSQL, Oracle, DB2*

পরীক্ষার ফাঁদ ✍ শেষ তিনটি ১. ***OODB* অবজেক্ট পরিচয় *(identity)* রাখে** — মানে সমান দুটি অবজেক্টও আলাদা অবজেক্ট — ঠিক যেটা রিলেশনাল কি ছুঁড়ে দেয়। ২. ***ORDB* মানে "ছবি জুড়ে দেওয়া রিলেশনাল ডেটাবেস" নয়**: এটি রিলেশনাল মডেলে **ব্যবহারকারী-নির্ধারিত টাইপ *(UDT)***, **নেস্টেড টেবিল**, **অবজেক্ট পরিচয়** ও **ইনহেরিট্যান্স** যোগ করে, আর রিলেশনাল মূল অক্ষত থাকে। ৩. "অবজেক্ট ডেটাবেস দ্রুত" কথাটি **গভীর ও জটিল অবজেক্ট গ্রাফের** ক্ষেত্রেই ধার্য; সাধারণ ব্যবসায়িক সারি ও জয়েনের ক্ষেত্রে পরিপক্ক রিলেশনাল বা অবজেক্ট-রিলেশনাল ইঞ্জিনই টুলিং ও খরচে এগিয়ে।

পরীক্ষার আগের রাতের শীট

তথ্যপাতা

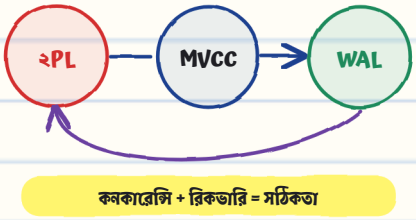
ACID = অ্যাটমিক ... | ... কনসিসটেন্সি, আইসোলেশন, ডিওরেবিলিটি
Commit = সব, Rollback = কিছু নয় | Active → পার্শিয়ালি কমিটেড
→ Committed → সমাপ্ত | Active → Failed → অ্যাবোর্টেড
সিরিয়াল = একটির পর আরেকটি | সিরিয়ালাইজেবল = সিরিয়ালের মতো
ডার্টি: অতি তাড়াতাড়ি | লস্ট আপডেট: অতি দেরিতে
নন-রিপিটেবল: মান বদলাল | ফ্যান্টম: সারি এল
S + S = সামঞ্জস্যপূর্ণ | S + X, X + X = সংঘর্ষ
২PL: আগে নাও, পরে ছাড়ো | ২PL ≠ ডেডলক-মুক্ত
স্ট্রিক্ট / রিগরাস ২PL ⇒ স্ট্রিক্ট | কনজারভেটিভ ⇒ ডেডলক-মুক্ত
ওয়েট-ফর চক্কর = ডেডলক | একটি অ্যাবোর্ট, বা নির্দিষ্ট ক্রম
টাইমস্ট্যাম্প: অ্যাবোর্ট হতে পারে | অপটিমিস্ট: কমিটে যাচাই
MVCC: স্ন্যাপশট পাঠক | RU < RC < RR < সিরিয়াল
মানদণ্ডে RR: ফ্যান্টম চলে | InnoDB / PG: ফ্যান্টম নেই
WAL: আগে লগ, পরে ডেটা | আনকমিটেডের UNDO, কমিটেডের REDO
ডিফার্ড ⇒ শুধু REDO | ইমিডিয়েট ⇒ UNDO + REDO
চেকপয়েন্ট ⇒ এর পরের লগ | ফুল / ডিফারেনশিয়াল / ইনক্রিমেন্টাল
DAC মালিক · MAC লেবেল | RBAC পদবি
GRANT দেয়, REVOKE কেড়ে | লিস্ট-প্রিভিলেজই নীতি
XPath = পাথ · XQuery = FLWOR | OODB: অবজেক্ট + OQL
ORDB: টেবিল + UDT | কনসিসটেন্সি দেয় কনস্ট্রেন্ট

→ MCQ-র জন্য সেরা ১২টি এক-লাইনার

1. ট্রানজেকশন সব-অথবা-কিছু-নয়
2. **কমিটেড শেষ অবস্থা নয়** — সমাপ্ত
3. **ব্যর্থ Active থেকে**; অ্যাবোর্ট ব্যর্থ থেকে
4. সিরিয়ালাইজেবল = কোনো সিরিয়ালের মতো
5. শুধু **S-এর সঙ্গে S** সামঞ্জস্যপূর্ণ
6. ২PL সিরিয়ালাইজেবল, **ডেডলক-মুক্ত নয়**
7. RR আটকায় মান বদলানো, **সারি আসা নয়**
8. MVCC আটকায় কেবল পাঠক-লেখক ব্লকিং
9. **WAL: আগে লগ রেকর্ড ডিস্কে**
10. ডিফার্ড ⇒ **শুধু REDO**; ইমিডিয়েট ⇒ দুটো
11. **DAC মালিক, MAC লেবেল, RBAC পদবি**
12. FLWOR কেবল **XQuery**-তে

সবচেয়ে বড় ফাঁদগুলো ২PL **ডেডলক আটকায়** দাবি করা · **পার্শিয়ালি কমিটেডের পরে ব্যর্থ** আঁকা · **ফ্যান্টমকে** বদলানো মান বলা · ইঞ্জিনের নাম না বলে **RR ফ্যান্টম আটকায়** বলা · ডেটা লেখার **পরে** লগ · কমিটেড কাজের **UNDO** · **চেকপয়েন্ট = পূর্ণ কপি** · **MAC কে DAC** ভাবা · **XPath-কে FLWOR** দেওয়া।

সব কৌশল একসঙ্গে কমিটির মতো ভাবো · সক্রিয় দিয়ে শুরু, সমাপ্তে শেষ · ডার্টি অতি তাড়াতাড়ি, লস্ট অতি দেরিতে · আগে দরজা খোলা, পরে বন্ধ · লক সার পড়ে, টাইমস্ট্যাম্প বিচার করে, অপটিমিস্ট আশা করে, সংস্করণ চালচা করে · বেশি আইসোলেশন = বেশি অপেক্ষা · আগে লগ, পরে ডেটা · আনকমিটেডের UNDO, কমিটেডের REDO · DAC মালিক, MAC লেবেল, RBAC পদবি।



তিনটি কৌশল, একটি প্রতিশ্রুতি

✓ অনুশীলনী MCQ (পরীক্ষার ধরনে, ১০ নম্বর)

আগে চেষ্টা করো, তারপর দেখো

১. একটি ট্রানজেকশন ***Committed*** অবস্থায় পৌঁছায় কখন?
ক) তার শেষ স্টেটমেন্ট চলার পর
খ) সব আপডেট ডেটা পেজে ফ্লাশ হলে
গ) তার কমিট লগ রেকর্ড স্থায়ী স্টোরেজে গেলে
ঘ) তার ধারিত সব লক ছাড়ার পর

২. কোন লক-জোড়া **সামঞ্জস্যপূর্ণ**?
ক) *S(A)* সঙ্গে *X(A)*
খ) *X(A)* সঙ্গে *S(A)*
গ) *S(A)* সঙ্গে *S(A)*
ঘ) *X(A)* সঙ্গে *X(A)*

৩. টু-ফেজ লকিং (*2PL*) কী নিশ্চয়তা দেয়?
ক) ডেডলক থেকে মুক্তি
খ) কনফ্লিক্ট-সিরিয়ালাইজেবল সিডিউল
গ) শুধু ক্যাসকেডিং রোলব্যাক থেকে মুক্তি
ঘ) কোনো ট্রানজেকশন কখনো অ্যাবোর্ট হয় না

৪. **ফ্যান্টম** ঘটে কখন?
ক) একটি সারির মান দুবার আপডেট হলে
খ) আনকমিটেড মান পড়া হলে
গ) দুটি আপডেট একে অপরকে উপরে লিখলে
ঘ) পুনরাবৃত্ত রেঞ্জ কোয়েরিতে নতুন সারি এলে

৫. *SQL* মানদণ্ড অনুযায়ী **রিপিটেবল রিড** এখনো কোনটি অনুমতি দেয়?
ক) ডার্টি রিড
খ) লস্ট আপডেট
গ) ফ্যান্টম রিড
ঘ) কিছুই নয় — এটি পুরো সিরিয়ালাইজেবল

৬. রাইট-অ্যাহেড লগিং নিয়ম কী দাবি করে?
ক) লগের আগে ডেটা পেজ লেখা হয়
খ) লগ রেকর্ড আগে স্থায়ী স্টোরেজে পৌঁছায়
গ) দুটো একই ডিস্ক ব্লকে লেখা হয়
ঘ) প্রতিটি কমিটের পর চেকপয়েন্ট

৭. **ইমিডিয়েট আপডেটে** ক্র্যাশ হলে রিকভারিতে কী করতে হয়?
ক) শুধু *REDO*
খ) শুধু *UNDO*
গ) আনকমিটেডের *UNDO* ও কমিটেডের *REDO*
ঘ) শুধু শেষ ফুল ব্যাকআপ থেকে *restore*

৮. **রাইট-স্কেউ** অসঙ্গতি কোন অবস্থায় সম্ভব?
ক) সিরিয়ালাইজেবল সিডিউলে
খ) স্ন্যাপশট আইসোলেশনে
গ) টু-ফেজ লকিংয়ে
ঘ) কনজারভেটিভ *2PL*-এ

৯. *MAC* সম্পর্কে কোন বক্তব্য সঠিক?
ক) মালিক যাকে খুশি গ্রান্ট করতে পারে
খ) অধিকার ঠিক করে সিস্টেমের সিকিউরিটি লেবেল
গ) অধিকার কাজের রোলের সঙ্গে যুক্ত
ঘ) ব্যবহারকারী শ্রেণিবিভাগ সবসময় নামাতে পারে

১০. ***for / where / return*** কোন ভাষার?
ক) *XPath*
খ) *XQuery*
গ) *SQL*
ঘ) *JSONPath*

উত্তরমালা → ১-গ · ২-গ · ৩-খ · ৪-ঘ · ৫-গ · ৬-খ · ৭-গ · ৮-খ · ৯-খ · ১০-খ

লিখিত প্রশ্ন ট্রানজেকশনের সংজ্ঞা দাও এবং চারটি *ACID* বৈশিষ্ট্য বলো, **প্রতিটি কে রক্ষা করে তা-সহ**। **লক সামঞ্জস্য ম্যাট্রিক্স** দাও ও **টু-ফেজ লকিং** ব্যাখ্যা করো, তারপর বলো *2PL* কী **দেয় না** এবং ডেডলক কীভাবে শনাক্ত হয়। ***WAL* নিয়ম** ও ক্র্যাশে **ডিফার্ড** ও **ইমিডিয়েট** আপডেটের পার্থক্য বলো। ১০ নম্বর

১ **ট্রানজেকশন ও *ACID* (৩)** — **একটি একক যৌক্তিক কাজ হিসেবে সম্পাদিত অপারেশনের ক্রম**, শেষে COMMIT বা ROLLBACK। **অ্যাটমিসিটি** সব অথবা কিছু নয়, রক্ষা করে রিকভারি ম্যানেজারের *UNDO*। **কনসিসটেন্সি** আগে ও পরে ঘোষিত ইন্টেগ্রিটি কনস্ট্রেন্ট মান্য — কনস্ট্রেন্টগুলো ব্যবহারকারীর, ডিবিএমএসের নয়। **আইসোলেশন** প্রতিটি ট্রানজেকশন একা চলছে বলে মনে হয়, রক্ষা করে কনকারেন্সি কন্ট্রোল। **ডিওরেবিলিটি** কমিটের পর ক্র্যাশেও টেকে থাকা, রক্ষা করে রিকভারি ম্যানেজার লগ ও *WAL* দিয়ে। ৩

২ **লক ম্যাট্রিক্স (২)** — S(A) সঙ্গে S(A) হলো **একমাত্র** সামঞ্জস্যপূর্ণ জোড়া; S সঙ্গে X, X সঙ্গে S এবং X সঙ্গে X — সবই সংঘর্ষ। অনেক পাঠক, এক লেখক, দুজন লেখক কখনো নয়। ২

৩ ***2PL* ও তার সীমা (২)** — **গ্রোয়িং পর্ব** লক অর্জন করে কিন্তু ছাড়ে না, **শ্রিংকিং পর্ব** ছাড়ে কিন্তু অর্জন করে না; সিডিউলটি **কনফ্লিক্ট-সিরিয়ালাইজেবল**, তবে বেসিক *2PL* কমিটের আগে লক ছাড়তে পারে, তাই রিকভারেবল হওয়া নিশ্চিত নয়; **স্ট্রিক্ট *2PL*** (*X* লক কমিট পর্যন্ত) ক্যাসকেডিং রোলব্যাক আটকায়। এটি **ডেডলক থেকে মুক্তি বা স্টারভেশন থেকে মুক্তি নিশ্চয়তা দেয় না**। ২

৪ **ডেডলক (১)** — **ওয়েট-ফর গ্রাফ** বানাও, $T_i \to T_j$ মানে T_i T_j-এর জন্য অপেক্ষা করছে; নিয়মিত **চক্কর** খোঁজা হয়; চক্কর মিললে একটি ট্রানজেকশন **অ্যাবোর্ট** করে আবার চালাতে হয়। প্রতিরোধে নির্দিষ্ট লক-ক্রম বা *wait-die / wound-wait* ব্যবহার হয়। ১

৫ ***WAL* ও দুটি আপডেট পলিসি (২)** — আপডেটের লগ রেকর্ড স্থায়ী স্টোরেজে আগে, তারপর সংশ্লিষ্ট ডেটা পেজ। **ডিফার্ড আপডেটে** কমিটের আগে কোনো পেজ লেখা হয় না, তাই রিকভারি করে **শুধু *REDO***; **ইমিডিয়েট আপডেটে** পেজ আগেই ফ্লাশ হয়, তাই **আনকমিটেডের *UNDO* ও কমিটেডের *REDO*** দুটোই। চেকপয়েন্ট থেকেই লগ পুনরাবৃত্তি শুরু হয়। ২

→ পরীক্ষার আগে ✓

- ✓ পাঁচটি ট্রানজেকশন-স্টেট ডায়াগ্রাম আঁকতে পারি
- ✓ পাঁচটি কনকারেন্সি অসঙ্গতির নাম বলতে পারি
- ✓ লক সামঞ্জস্য ম্যাট্রিক্স হুবহু বলতে পারি
- ✓ *2PL* কী দেয় *না* তা বলতে পারি
- ✓ *WAL* নিয়ম শব্দে শব্দে বলতে পারি
- ✓ *XPath* থেকে *XQuery*, *OODB* থেকে *ORDB* আলাদা করতে পারি